

SOFTWARE TEST PLAN

Project Name: Workspace-Vrithub

Software Testing: Manual Testing

Application Type: Web Application

Version: 1.0

Prepared by: Harshal Dilip Suryawanshi

1. INTRODUCTION

1.1 Purpose

This Software Test Plan describes the testing activities planned for the **Workspace-Vrithub** web application.

Workspace-Vrithub is an internship platform designed to connect students/candidates with companies/recruiters. The application provides separate functionality for candidates and recruiters. Candidates can register, log in, provide their details and apply for internships. Companies can register, log in, provide company details and post internship opportunities.

The application uses **Node.js with Express.js** for the backend and **PostgreSQL** for storing application data. Authentication is implemented using **Passport.js**, including local username/password authentication and Google OAuth.

The purpose of this test plan is to define the modules to be tested, testing approach, test cases, test data, defect handling process, risks and criteria for completing testing.

The structure follows the same type of test-plan organization used in the provided sample, including objectives, scope, testing types, environment, test data, entry/exit criteria and defect management.

2. TEST OBJECTIVES

The main objectives of testing Workspace-Vrithub are:

1. To verify that candidate registration works correctly.
2. To verify candidate login using email and password.
3. To verify company/recruiter registration.
4. To verify company/recruiter login.
5. To verify Google OAuth authentication.
6. To verify session-based authentication.

7. To verify student details are stored correctly.
 8. To verify company details are stored correctly.
 9. To verify student profile information can be updated.
 10. To verify company profile information can be updated.
 11. To verify companies can post internship opportunities.
 12. To verify students can apply for internships.
 13. To verify internship information is stored correctly in PostgreSQL.
 14. To verify invalid input is handled properly.
 15. To verify unauthorized users cannot access protected pages.
 16. To verify database errors are handled without crashing the application.
 17. To identify defects before the application is used in a real environment.
 18. To perform regression testing after important changes.
-

3. SCOPE OF TESTING

3.1 In Scope

The following functionalities are included in testing:

- Home page
- Candidate registration
- Candidate login
- Candidate logout
- Candidate session handling
- Google authentication for candidates
- Student details
- Student details retrieval
- Student details update
- Company/recruiter registration
- Company/recruiter login
- Company logout
- Company session handling

- Google authentication for recruiters
 - Company details
 - Company details retrieval
 - Company details update
 - Internship posting
 - Internship listing
 - Internship application
 - PostgreSQL database operations
 - Password hashing
 - Input validation
 - Authentication
 - Authorization
 - Error handling
 - HTTP status codes
-

3.2 Out of Scope

The following areas are outside the main scope of this testing plan:

- Internal testing of Google's OAuth servers
 - Internal testing of PostgreSQL itself
 - Testing third-party libraries such as Passport.js and bcrypt internally
 - Testing internet service provider performance
 - Large-scale production load testing
 - Professional penetration testing
 - Testing infrastructure outside the project
 - Features that are not implemented in the supplied project files
-

4. APPLICATION OVERVIEW

Workspace-Vrithub works as an internship platform with two main types of users:

1. Candidate/Student

A student can:

- Register
- Log in
- Authenticate using Google
- Provide personal/academic details
- View student details
- Update student information
- Apply for an internship

2. Company/Recruiter

A company can:

- Register
- Log in
- Authenticate using Google
- Provide company details
- Update company information
- Post an internship
- View posted internships

The basic application flow can be represented as:

Candidate

Register → Login → Student Details → View/Update Profile → Apply for Internship

Company

Register → Login → Company Details → View/Update Profile → Post Internship

Both flows use the PostgreSQL database for storing application information.

5. TECHNOLOGY USED

Component	Technology
Backend	Node.js
Web Framework	Express.js

Database	PostgreSQL
Authentication	Passport.js
Local Authentication	Passport Local Strategy
Google Login	Passport Google OAuth2
Password Security	bcrypt
Sessions	express-session
Request Body Processing	body-parser
File Upload Support	multer
Environment Configuration	dotenv
Template Engine	EJS
Version Control	Git/GitHub

6. APPLICATION MODULES

6.1 Candidate Registration Module

The candidate registration functionality uses the /register route.

The candidate provides:

- Email/username
- Password

The password is hashed using bcrypt before it is stored in the database.

Testing areas

- Valid registration
 - Duplicate email
 - Empty email
 - Empty password
 - Invalid email
 - Password handling
 - Database insertion
 - Session creation after registration
-

7. CANDIDATE LOGIN MODULE

The candidate login functionality uses the /login route and Passport Local Strategy.

The system checks the candidate's email against the database and compares the entered password with the stored hashed password.

Testing areas

- Correct email and password
 - Wrong password
 - Unregistered email
 - Empty credentials
 - Session creation
 - Failed login
 - Redirect after successful login
 - Redirect after failed login
-

8. GOOGLE AUTHENTICATION MODULE

The project supports Google OAuth authentication through Passport Google OAuth2.

The flow includes:

/auth/google

followed by the callback:

/auth/google/explore

Testing areas

- Google login with valid account
- Existing Google account
- New Google account
- OAuth callback
- Failed authentication
- Session creation
- Redirect after authentication
- Missing Google credentials/configuration

The internal operation of Google's authentication service is not part of the testing scope.

9. STUDENT DETAILS MODULE

Students can submit their personal and academic information.

The student details include:

- Name
- Email
- Qualification
- Contact number
- Location
- College name
- Skills
- Achievements
- Interested internship/domain

The data is stored in the students PostgreSQL table.

Testing areas

- Valid student details
 - Missing fields
 - Duplicate email
 - Duplicate contact number
 - Student details retrieval
 - Invalid email in URL
 - Non-existing student
 - Database failure
-

10. STUDENT PROFILE UPDATE MODULE

The project contains functionality for updating student details using email.

The update operation uses a PATCH request.

The fields that can be updated include:

- Name
- Qualification
- Contact number
- Location
- College name
- Skills
- Achievements
- Interested internship

Testing areas

- Update one field
 - Update multiple fields
 - Update all fields
 - Empty PATCH request
 - Invalid email
 - Non-existing student
 - Duplicate contact number
 - Database failure
-

11. COMPANY REGISTRATION MODULE

Companies/recruiters register using the /register2 route.

The system checks whether the email already exists in the recruiter table before creating the account.

Passwords are hashed using bcrypt.

Testing areas

- Valid company registration
- Duplicate email
- Empty email
- Empty password
- Invalid email

- Password hashing
 - Database insertion
 - Session creation
-

12. COMPANY LOGIN MODULE

Company login uses /login2 and Passport Local Strategy.

After successful login, the recruiter is redirected to the company exploration page.

Testing areas

- Valid login
 - Wrong password
 - Unknown email
 - Empty credentials
 - Successful redirect
 - Failed redirect
 - Session creation
-

13. COMPANY DETAILS MODULE

Companies can store their information using /companydetails.

The company information includes:

- Company name
- Qualification required
- Contact number
- Position name
- Skills required
- Job description
- Email
- Location
- Interested domain

The information is stored in the company table.

Testing areas

- Valid company details
 - Missing details
 - Duplicate email
 - Duplicate contact number
 - Valid database insertion
 - Database failure
 - Retrieval of company details
-

14. COMPANY PROFILE UPDATE MODULE

Company details can be updated using a PATCH operation.

The fields include:

- Company name
- Qualification required
- Contact number
- Position name
- Skills required
- Job description
- Location
- Interested domain

Testing areas

- Update one field
 - Update multiple fields
 - Empty update
 - Invalid email
 - Non-existing company
 - Duplicate contact number
 - Database error
-

15. INTERNSHIP POSTING MODULE

Companies can post internship opportunities using /postinternship.

The internship information includes:

- Company name
- Qualification required
- Contact number
- Position name
- Skills required
- Job description
- Email
- Location
- Interested domain

The data is stored in the internship_post table.

Testing areas

- Valid internship posting
 - Missing fields
 - Invalid data
 - Database insertion
 - Duplicate email
 - Successful response
 - Database failure
-

16. INTERNSHIP LISTING MODULE

The project provides /postedinternship for retrieving internship posts.

The endpoint retrieves records from the internship_post table.

Testing areas

- Retrieve available internships
- Empty internship table
- Database failure

- Response format
 - Multiple internship records
 - Correct internship information
-

17. INTERNSHIP APPLICATION MODULE

Students can apply for internships through /applyforinternship.

Application information includes:

- Student name
- Qualification
- Contact number
- College name
- Skills and achievements
- Bio
- Email
- Location
- Preferred internship location

The application is stored in the internship_application table.

Testing areas

- Valid application
 - Missing fields
 - Duplicate application email
 - Invalid information
 - Successful insertion
 - Database failure
 - Correct success response
-

18. DATABASE MODULE

PostgreSQL is used as the main database.

The supplied SQL file defines the following tables:

1. candidate
2. students
3. recruiter
4. company
5. internship_application
6. internship_post

Database testing will verify:

- Correct record insertion
 - Record retrieval
 - Record updates
 - Unique constraints
 - Primary keys
 - Required fields
 - Data consistency
 - Database connection
 - Handling of database errors
-

19. AUTHENTICATION AND SESSION TESTING

The project uses:

- Passport.js
- Passport Local Strategy
- Passport Google OAuth2
- express-session
- bcrypt

Testing will verify:

1. Valid users can log in.
2. Invalid users cannot log in.
3. Passwords are not stored as plain text for local registration.
4. Successful login creates a session.

5. Authenticated users can access protected functionality.
 6. Logout removes the active authentication session.
 7. Unauthenticated users are redirected where applicable.
-

20. TYPES OF TESTING

20.1 Functional Testing

Functional testing verifies that each feature performs its intended operation.

Examples:

- Registration
 - Login
 - Profile creation
 - Profile update
 - Internship posting
 - Internship application
-

20.2 Integration Testing

Integration testing verifies communication between different components.

Examples:

- Express.js + PostgreSQL
 - Passport + PostgreSQL
 - bcrypt + candidate authentication
 - bcrypt + recruiter authentication
 - Application form + database
 - Internship form + database
-

20.3 System Testing

The complete application will be tested from the user's point of view.

Example:

Candidate Registration → Login → Add Details → View Profile → Apply for Internship

Another flow:

Company Registration → Login → Add Company Details → Post Internship → View Internship

20.4 Security Testing

Basic security testing will cover:

- Password hashing
 - Authentication
 - Session handling
 - Unauthorized access
 - SQL injection attempts
 - Invalid user input
 - Duplicate accounts
 - Session logout
-

20.5 Negative Testing

Negative testing checks how the application responds to incorrect inputs.

Examples:

- Empty email
 - Invalid email
 - Empty password
 - Incorrect password
 - Duplicate email
 - Invalid profile email
 - Missing internship information
 - Missing application information
 - Invalid database input
-

20.6 Regression Testing

Regression testing will be performed after changes to make sure existing functionality continues to work.

For example, changes to candidate registration should not affect:

- Candidate login
- Student details
- Internship applications

Similarly, changes to company registration should not affect:

- Company login
- Company details
- Internship posting

21. TESTING APPROACH

Testing will be performed in the following sequence:

Step 1 – Understand Requirements

Study the application functionality and identify important user flows.

Step 2 – Prepare Test Scenarios

Identify valid, invalid and boundary conditions.

Step 3 – Prepare Test Cases

Each test case will contain:

- Test Case ID
- Module
- Test Scenario
- Test Data
- Expected Result
- Actual Result
- Status

Step 4 – Execute Tests

Test the application through the available web routes/API requests.

Step 5 – Record Defects

If actual and expected results are different, the issue will be recorded.

Step 6 – Fix and Retest

After the defect is fixed, the same test case will be executed again.

Step 7 – Regression Testing

Related modules will be tested after important fixes.

Step 8 – Test Closure

Testing will be completed after satisfying the exit criteria.

22. TEST ENVIRONMENT

Component	Environment
Operating System	Windows 10/11
Runtime	Node.js
Framework	Express.js
Database	PostgreSQL
Browser	Google Chrome
Database Tool	PostgreSQL / pgAdmin
Code Editor	VS Code
Authentication	Passport.js
Version Control	Git/GitHub
API Testing	Postman / Browser
Template Engine	EJS

The project uses port **3000** for the Express server.

23. TEST DATA

23.1 Candidate Test Data

Field	Valid Example	Invalid Example
Email	student@gmail.com	student
Password	Student@123	Blank

Name	Rahul Patil	Blank
Qualification	B.E. Computer Engineering	Blank
Contact	9876543210	Invalid text
College	ABC College	Blank
Skills	Java, SQL	—
Location	Pune	—

23.2 Company Test Data

Field	Valid Example	Invalid Example
Email	company@gmail.com	company
Password	Company@123	Blank
Company Name	ABC Technologies	Blank
Position	Software Intern	—
Qualification	B.E./B.Tech	—
Contact	9876543210	Invalid value
Location	Pune	—
Domain	Software Development	—

23.3 Internship Test Data

Field	Sample Value
Company Name	ABC Technologies
Position	Java Developer Intern
Qualification	B.E. CSE
Skills	Java, SQL, Spring Boot
Location	Pune
Interested Domain	Software Development
Job Description	Backend development internship

24. CANDIDATE REGISTRATION TEST CASES

TC ID	Test Case	Expected Result
TC-CAN-01	Register with valid email and password	Candidate should be registered
TC-CAN-02	Register with existing email	Duplicate account should not be created
TC-CAN-03	Register with empty email	Registration should fail
TC-CAN-04	Register with empty password	Registration should fail
TC-CAN-05	Register with invalid email	Invalid input should be handled
TC-CAN-06	Register with valid password	Password should be securely stored
TC-CAN-07	Check database after registration	Candidate record should exist
TC-CAN-08	Register when database is unavailable	Application should handle error

25. CANDIDATE LOGIN TEST CASES

TC ID	Test Case	Expected Result
TC-LOGIN-01	Login with valid credentials	Login should be successful
TC-LOGIN-02	Login with wrong password	Login should fail
TC-LOGIN-03	Login with unknown email	Login should fail
TC-LOGIN-04	Login with empty email	Request should be rejected/handled
TC-LOGIN-05	Login with empty password	Request should be rejected/handled
TC-LOGIN-06	Successful login	User session should be created
TC-LOGIN-07	Failed login	User should not be authenticated
TC-LOGIN-08	Logout after login	User session should be terminated

26. STUDENT DETAILS TEST CASES

TC ID	Test Case	Expected Result
TC-STU-01	Submit valid student details	Details should be stored
TC-STU-02	Submit duplicate email	Database should reject duplicate email
TC-STU-03	Submit duplicate contact number	Database should reject duplicate contact
TC-STU-04	Submit without name	System should handle missing data
TC-STU-05	Submit without qualification	Request should be handled
TC-STU-06	Retrieve existing student	Student details should be displayed
TC-STU-07	Retrieve non-existing student	404/error response should be returned
TC-STU-08	Use invalid email in URL	Appropriate error should be returned

27. STUDENT UPDATE TEST CASES

TC ID	Test Case	Expected Result
TC-UPD-STU-01	Update student name	Name should be updated
TC-UPD-STU-02	Update multiple fields	Selected fields should be updated
TC-UPD-STU-03	Update all supported fields	Details should be updated
TC-UPD-STU-04	Send empty PATCH request	400 error should be returned
TC-UPD-STU-05	Update non-existing student	Appropriate response should be returned
TC-UPD-STU-06	Update duplicate contact	Database constraint should be handled

28. COMPANY REGISTRATION TEST CASES

TC ID	Test Case	Expected Result
TC-COM-01	Register company with valid data	Company account should be created
TC-COM-02	Register using existing email	Duplicate account should not be created
TC-COM-03	Register with empty email	Registration should fail
TC-COM-04	Register with empty password	Registration should fail
TC-COM-05	Register with invalid email	Invalid input should be handled
TC-COM-06	Verify password storage	Password should be hashed
TC-COM-07	Verify database record	Recruiter record should exist

29. COMPANY LOGIN TEST CASES

TC ID	Test Case	Expected Result
TC-CLOGIN-01	Login using valid credentials	Company should log in
TC-CLOGIN-02	Login using wrong password	Login should fail
TC-CLOGIN-03	Login using unknown email	Login should fail
TC-CLOGIN-04	Login with empty credentials	Login should be rejected/handled
TC-CLOGIN-05	Successful login	Session should be created
TC-CLOGIN-06	Logout	Company session should end

30. COMPANY DETAILS TEST CASES

TC ID	Test Case	Expected Result
TC-CDET-01	Submit valid company details	Details should be stored
TC-CDET-02	Submit duplicate email	Database should reject duplicate
TC-CDET-03	Submit duplicate contact	Database should reject duplicate
TC-CDET-04	Retrieve existing company	Company details should be returned
TC-CDET-05	Retrieve non-existing company	404/error should be returned
TC-CDET-06	Submit missing company name	System should handle missing data

31. COMPANY UPDATE TEST CASES

TC ID	Test Case	Expected Result
TC-CUPD-01	Update company name	Name should be updated
TC-CUPD-02	Update position	Position should be updated
TC-CUPD-03	Update multiple fields	Selected fields should be updated
TC-CUPD-04	Send empty PATCH request	400 error should be returned

TC-CUPD-05	Update non-existing company	Appropriate response should be returned
TC-CUPD-06	Update duplicate contact number	Database error should be handled

32. INTERNSHIP POSTING TEST CASES

TC ID	Test Case	Expected Result
TC-INT-01	Post internship with valid data	Internship should be posted
TC-INT-02	Post internship without company name	Request should be handled
TC-INT-03	Post internship without position	Request should be handled
TC-INT-04	Post internship with valid qualification	Data should be stored
TC-INT-05	Post internship with valid skills	Skills should be stored
TC-INT-06	Verify database record	Internship should exist in database
TC-INT-07	Database unavailable while posting	Appropriate error should be returned

33. INTERNSHIP LISTING TEST CASES

TC ID	Test Case	Expected Result
TC-LIST-01	Retrieve internship list	Internship records should be returned
TC-LIST-02	Retrieve when no internships exist	Empty result should be handled
TC-LIST-03	Retrieve multiple internships	All available records should be returned
TC-LIST-04	Database unavailable	Error response should be returned

34. INTERNSHIP APPLICATION TEST CASES

TC ID	Test Case	Expected Result
TC-APP-01	Apply with valid student information	Application should be submitted
TC-APP-02	Apply with missing student name	Request should be handled
TC-APP-03	Apply with missing email	Database/application should reject invalid data

TC-APP-04	Apply with duplicate email	Database constraint should be handled
TC-APP-05	Submit valid skills and achievements	Data should be stored
TC-APP-06	Submit valid internship location	Data should be stored
TC-APP-07	Verify database after application	Application record should exist
TC-APP-08	Database unavailable	Internal error should be handled

35. GOOGLE OAUTH TEST CASES

TC ID	Test Case	Expected Result
TC-OAUTH-01	Start Google login	Google authentication page should open
TC-OAUTH-02	Login using valid Google account	User should be authenticated
TC-OAUTH-03	Existing Google user	Existing database record should be used
TC-OAUTH-04	New Google user	New user should be created
TC-OAUTH-05	OAuth authentication failure	User should be redirected appropriately
TC-OAUTH-06	Missing OAuth configuration	Application should handle configuration failure

36. DATABASE TESTING

Database testing will verify the tables defined in the project SQL file.

Candidate Table

candidate

Used for candidate authentication.

Student Table

students

Used for student profile information.

Recruiter Table

recruiter

Used for company authentication.

Company Table

company

Used for company information.

Internship Application Table

internship_application

Used for storing internship applications.

Internship Post Table

internship_post

Used for storing internships posted by companies.

The following database conditions will be tested:

- Primary key constraints
- Unique email constraints
- Unique contact constraints
- NOT NULL fields
- Correct data insertion
- Correct data retrieval
- Correct update operations
- Duplicate records
- Database connection failures

37. API/ROUTE TESTING

The major application routes to be tested include:

Method	Route	Purpose
GET	/	Home page

GET	/login	Candidate login page
GET	/register	Candidate registration page
POST	/register	Candidate registration
POST	/login	Candidate login
GET	/logout	Candidate logout
GET	/auth/google	Candidate Google authentication
GET	/studentdetails/:email	Retrieve student details
POST	/studentdetails	Store student details
POST	/applyforinternship	Submit internship application
GET	/login2	Company login
GET	/register2	Company registration
POST	/register2	Company registration
POST	/login2	Company login
GET	/logout2	Company logout
GET	/companydetails/:email*	Retrieve company details
POST	/companydetails	Store company details
POST	/postinternship	Post internship
GET	/postedinternship	Retrieve internship posts

*The exact route should be verified during execution because route definitions and source formatting should be checked against the running application before final test execution.

38. ENTRY CRITERIA

Testing can begin when:

1. Source code is available.
2. Node.js is installed.
3. Project dependencies are installed.
4. PostgreSQL is installed and running.
5. Required database tables are created.
6. Environment variables are configured.

7. The Express server starts successfully.
 8. Required authentication configuration is available.
 9. Test data has been prepared.
-

39. EXIT CRITERIA

Testing can be considered complete when:

1. All planned test cases have been executed.
2. Critical defects have been fixed.
3. Major functional defects have been resolved or accepted.
4. Regression testing has been completed.
5. Database operations work correctly.
6. Authentication functionality works correctly.
7. Major candidate and recruiter flows have been verified.
8. No unresolved critical defects remain.
9. Test results have been documented.
10. Final testing summary has been prepared.

The sample test plan also uses defined entry and exit criteria to establish when testing starts and when it can be closed.

40. DEFECT MANAGEMENT

Any issue found during testing will be recorded as a defect.

The defect report should contain:

- Defect ID
- Module
- Test Case ID
- Date
- Description
- Steps to reproduce
- Expected result

- Actual result
- Severity
- Priority
- Status
- Assigned person
- Resolution
- Retest result

Defect Life Cycle

New → Assigned → Open → Fixed → Retest → Closed

If the problem is still present:

Retest → Reopened → Fixed → Retest

41. DEFECT SEVERITY

Severity Description

Critical Major application functionality is unavailable

High Important functionality is affected

Medium Functionality is affected but a workaround exists

Low Minor issue with little effect on the application

The severity classification follows the general approach used in the sample test plan.

42. DEFECT PRIORITY

Priority Description

P1 – High Should be fixed immediately

P2 – Medium Should be fixed before final release if possible

P3 – Low Can be fixed in a later version

43. RISK MANAGEMENT

Risk	Impact	Mitigation
PostgreSQL unavailable	High	Verify database connection before testing
Incorrect database configuration	High	Check environment variables
Google OAuth configuration failure	High	Verify OAuth credentials
Session configuration problem	High	Test login/logout carefully
Duplicate database records	Medium	Test unique constraints
Invalid user input	Medium	Perform negative testing
Authentication failure	High	Test valid and invalid credentials
Server crash	High	Check application logs
Database constraint violation	Medium	Test duplicate and missing data
Network failure during OAuth	Medium	Test authentication failure handling

The sample test plan also considers technical, environment and external-service risks during planning.

44. TEST DELIVERABLES

The following testing documents/results will be prepared:

1. Software Test Plan
2. Test Scenarios
3. Test Cases
4. Test Data
5. Test Execution Report
6. Defect Reports
7. Retesting Report
8. Regression Testing Report
9. Requirement Traceability Matrix
10. Test Summary Report

These are consistent with the deliverable-oriented structure of the provided sample.

45. REQUIREMENT TRACEABILITY MATRIX

Requirement	Module	Test Cases
Candidate registration	Candidate	TC-CAN-01 to TC-CAN-08
Candidate login	Authentication	TC-LOGIN-01 to TC-LOGIN-08
Student details	Student	TC-STU-01 to TC-STU-08
Student profile update	Student	TC-UPD-STU-01 to TC-UPD-STU-06
Company registration	Company	TC-COM-01 to TC-COM-07
Company login	Company	TC-CLOGIN-01 to TC-CLOGIN-06
Company details	Company	TC-CDET-01 to TC-CDET-06
Company profile update	Company	TC-CUPD-01 to TC-CUPD-06
Internship posting	Internship	TC-INT-01 to TC-INT-07
Internship listing	Internship	TC-LIST-01 to TC-LIST-04
Internship application	Application	TC-APP-01 to TC-APP-08
Google authentication	OAuth	TC-OAUTH-01 to TC-OAUTH-06

Traceability ensures that the main requirements have corresponding test coverage.

46. END-TO-END TEST SCENARIO

Scenario 1 – Student Applies for Internship

Step 1

Student opens the application.

Expected: Home page should be displayed.

Step 2

Student registers with a new email and password.

Expected: Candidate account should be created.

Step 3

Student logs in.

Expected: Student should be authenticated and redirected to the appropriate page.

Step 4

Student enters personal and academic details.

Expected: Student information should be stored in PostgreSQL.

Step 5

Student views available internships.

Expected: Available internship records should be displayed.

Step 6

Student submits an internship application.

Expected: Application should be stored successfully.

47. END-TO-END TEST SCENARIO

Scenario 2 – Company Posts Internship

Step 1

Company opens the recruiter registration page.

Step 2

Company registers using email and password.

Expected: Recruiter account should be created.

Step 3

Company logs in.

Expected: Company should be authenticated.

Step 4

Company enters company details.

Expected: Company details should be stored.

Step 5

Company creates an internship post.

Expected: Internship should be inserted into the internship_post table.

Step 6

Internship listing is opened.

Expected: Posted internship should be available in the internship list.

48. TEST EXECUTION REPORT FORMAT

Actual execution results should be entered after running the application.

Test Case ID	Expected Result	Actual Result	Status
TC-CAN-01	Candidate registered	To be tested	—
TC-LOGIN-01	Login successful	To be tested	—
TC-STU-01	Student details stored	To be tested	—
TC-COM-01	Company registered	To be tested	—
TC-CLOGIN-01	Company login successful	To be tested	—
TC-INT-01	Internship posted	To be tested	—
TC-APP-01	Application submitted	To be tested	—

Note: Pass/Fail should be filled only after actual execution.

49. TESTING TOOLS

Tool	Purpose
VS Code	Code inspection and development
Google Chrome	Web application testing
Postman	API/route testing
PostgreSQL	Database
pgAdmin	Database verification
Git/GitHub	Source code management
Node.js	Running the application
npm	Dependency management

50. TESTING RESPONSIBILITIES

Activity	Responsibility
Test planning	Tester
Test case preparation	Tester
Test data preparation	Tester
Test execution	Tester

Defect identification	Tester
Defect fixing	Developer
Retesting	Tester
Regression testing	Tester
Final review	Project Reviewer

51. APPROVAL

The test plan will be reviewed before the testing process begins.

Name	Status
Harshal Dilip Suryawanshi	Prepared

52. CONCLUSION

This Software Test Plan provides a structured approach for testing the **Workspace-VrittiHub** web application.

The testing covers the major functions of the project, including candidate registration and login, recruiter registration and login, Google authentication, student and company profiles, internship posting, internship listing and internship applications.

Database testing will also be performed to verify that student, recruiter, company, internship and application information is stored and retrieved correctly.

Functional, integration, system, security, negative and regression testing will be used to identify problems in the application.

The main objective is to make sure that both students and companies can use the important features of Workspace-VrittiHub correctly and that the application handles invalid inputs and errors properly before final use.